

Three-Player, Three-Level Benchmark with Inequalities as the Innermost Preference

This is the exact problem solved by `test/quasi-vs.reduced.preference.inequality.jl`. It is built in two steps:

1. The call

```
build_benchmark_problem(; num_players = 3, levels = 3, kind)
```

in `test/benchmark_problems.jl` constructs a three-level generalized Nash problem with *hard* coupled inequality constraints.

2. The helper `innermost_preference_inequality_goop` then deletes those hard constraints and re-attaches each player's inequality vector as a *fourth, highest-priority preference level*, flagged with

```
is_prioritized_constraint = true.
```

This is the encoding that `experiments/robotic_arm_core.jl` uses.

Everything below is fully expanded — no symbolic shorthand for the constants.

1. Notation

There are three players. Player $p \in \{1, 2, 3\}$ owns a decision vector

$$x_p = (x_{p,1}, x_{p,2}, x_{p,3}, x_{p,4}, x_{p,5}) \in \mathbb{R}^5, \quad x = (x_1, x_2, x_3) \in \mathbb{R}^{15}.$$

In code, $x_{p,j}$ is `x[Block(p)][j]`. Write x_{-p} for the other two players' vectors, which player p treats as fixed parameters — the three player problems hold simultaneously, i.e. this is a generalized Nash equilibrium problem. There are parameters $\theta \in \mathbb{R}^3$, but every objective and constraint below ignores them; they are identically zero in the test.

Preference ordering. Preferences are stored `[outermost, ..., innermost]`, and the *innermost* is the *highest priority*. Player p has four levels:

level ℓ	controls	role	priority
1	$x_{p,5}$	objective	lowest
2	$x_{p,3}, x_{p,4}$	objective	
3	$x_{p,1}, x_{p,2}$	objective	
4	$h_p(x) \geq 0$	prioritized constraint	highest

Each level's objective still ranges over the *whole* vector x_p ; the coordinate lists say which coordinates that level's objective actually depends on.

2. The prioritized-constraint level

A level flagged `is_prioritized_constraint` does *not* enter as a constraint. It enters as the smooth penalty `ReducedG00P.smooth_piecewise_preference_objective`, applied elementwise and summed. In code that function is `ifelse(h ≥ 0, 0.0, (0 - h)^(level + 2))`, i.e.

$$\varphi(h, \ell) = \begin{cases} 0, & h \geq 0, \\ (-h)^{\ell+2}, & h < 0. \end{cases}$$

The inequality sits at $\ell = 4$, so the exponent is $\ell + 2 = 6$:

$$P_p(x) = \sum_{m=1}^4 \varphi(h_{p,m}(x), 4) = \sum_{m=1}^4 \max(-h_{p,m}(x), 0)^6.$$

We have $P_p \geq 0$ always, and $P_p(x) = 0$ iff every $h_{p,m}(x) \geq 0$. So this level's argmin set is exactly the feasible set of the original hard constraints, whenever that set is nonempty.

3. The nested problem

Player p solves the outermost level subject to being optimal for every higher-priority level:

$$\begin{aligned} \min_{x_p \in \mathbb{R}^5} \quad & J_p^{(1)}(x_p) \\ \text{subject to} \quad & x_p \in \mathcal{S}_p^{(2)}(x_{-p}), \end{aligned}$$

where each level's solution set is defined by the next one down:

$$\begin{aligned} \mathcal{S}_p^{(2)}(x_{-p}) &= \arg \min_{y \in \mathbb{R}^5} J_p^{(2)}(y) \quad \text{subject to} \quad y \in \mathcal{S}_p^{(3)}(x_{-p}), \\ \mathcal{S}_p^{(3)}(x_{-p}) &= \arg \min_{y \in \mathbb{R}^5} J_p^{(3)}(y) \quad \text{subject to} \quad y \in \mathcal{S}_p^{(4)}(x_{-p}), \\ \mathcal{S}_p^{(4)}(x_{-p}) &= \arg \min_{y \in \mathbb{R}^5} P_p(y, x_{-p}). \end{aligned}$$

The innermost problem is *unconstrained* — that is the whole point of the transformation. There are no hard constraints anywhere in this problem.

4. Coupling and constraint data

The coupled resource consumed by player p in coordinate j is

$$c_{p,j}(x) = x_{p,j} + 0.25 \sum_{k \neq p} x_{k,j},$$

with coupling weight 0.25 (`COUPLING_WEIGHT`). Each player has four inequality rows:

$$h_p(x) = \begin{pmatrix} C_{p,1} - c_{p,1}(x) \\ C_{p,2} - c_{p,2}(x) \\ c_{p,3}(x) - F_{p,3} \\ c_{p,4}(x) - F_{p,4} \end{pmatrix} \geq 0.$$

Rows 1 and 3 are *active* at the designed solution; rows 2 and 4 carry slack 1.

p	$C_{p,1}$	$C_{p,2}$	$F_{p,3}$	$F_{p,4}$	p	$t_{p,1}$	$t_{p,2}$	$t_{p,3}$	$t_{p,4}$	$t_{p,5}$
1	0.6375	2.4250	0.8625	0.8750	1	1.00	0.90	0.15	1.20	0.80
2	0.6750	2.5000	0.9000	0.9500	2	1.05	1.00	0.20	1.30	0.90
3	0.7125	2.5750	0.9375	1.0250	3	1.10	1.10	0.25	1.40	1.00

The right-hand table lists the unconstrained objective targets t_p — the point each objective would pick with no constraints.

5. kind = :quadratic, fully expanded

Objectives are squared distances to the targets; the constraint rows are affine.

Player 1

$$\begin{aligned}
& \min_{x_1 \in \mathbb{R}^5} (x_{1,5} - 0.80)^2 \\
& \text{subject to } x_1 \in \mathcal{S}_1^{(2)}(x_2, x_3) \\
& \mathcal{S}_1^{(2)} = \arg \min_y (y_3 - 0.15)^2 + (y_4 - 1.20)^2 \quad \text{s.t. } y \in \mathcal{S}_1^{(3)}, \\
& \mathcal{S}_1^{(3)} = \arg \min_y (y_1 - 1.00)^2 + (y_2 - 0.90)^2 \quad \text{s.t. } y \in \mathcal{S}_1^{(4)}, \\
& \mathcal{S}_1^{(4)} = \arg \min_y \left[\max(-(0.6375 - y_1 - 0.25x_{2,1} - 0.25x_{3,1}), 0)^6 \right. \\
& \quad + \max(-(2.4250 - y_2 - 0.25x_{2,2} - 0.25x_{3,2}), 0)^6 \\
& \quad + \max(-(y_3 + 0.25x_{2,3} + 0.25x_{3,3} - 0.8625), 0)^6 \\
& \quad \left. + \max(-(y_4 + 0.25x_{2,4} + 0.25x_{3,4} - 0.8750), 0)^6 \right].
\end{aligned}$$

Player 2

$$\begin{aligned}
& \min_{x_2 \in \mathbb{R}^5} (x_{2,5} - 0.90)^2 \\
& \text{subject to } x_2 \in \mathcal{S}_2^{(2)}(x_1, x_3) \\
& \mathcal{S}_2^{(2)} = \arg \min_y (y_3 - 0.20)^2 + (y_4 - 1.30)^2 \quad \text{s.t. } y \in \mathcal{S}_2^{(3)}, \\
& \mathcal{S}_2^{(3)} = \arg \min_y (y_1 - 1.05)^2 + (y_2 - 1.00)^2 \quad \text{s.t. } y \in \mathcal{S}_2^{(4)}, \\
& \mathcal{S}_2^{(4)} = \arg \min_y \left[\max(-(0.6750 - y_1 - 0.25x_{1,1} - 0.25x_{3,1}), 0)^6 \right. \\
& \quad + \max(-(2.5000 - y_2 - 0.25x_{1,2} - 0.25x_{3,2}), 0)^6 \\
& \quad + \max(-(y_3 + 0.25x_{1,3} + 0.25x_{3,3} - 0.9000), 0)^6 \\
& \quad \left. + \max(-(y_4 + 0.25x_{1,4} + 0.25x_{3,4} - 0.9500), 0)^6 \right].
\end{aligned}$$

Player 3

$$\begin{aligned}
& \min_{x_3 \in \mathbb{R}^5} (x_{3,5} - 1.00)^2 \\
& \text{subject to } x_3 \in \mathcal{S}_3^{(2)}(x_1, x_2) \\
& \mathcal{S}_3^{(2)} = \arg \min_y (y_3 - 0.25)^2 + (y_4 - 1.40)^2 \quad \text{s.t. } y \in \mathcal{S}_3^{(3)}, \\
& \mathcal{S}_3^{(3)} = \arg \min_y (y_1 - 1.10)^2 + (y_2 - 1.10)^2 \quad \text{s.t. } y \in \mathcal{S}_3^{(4)}, \\
& \mathcal{S}_3^{(4)} = \arg \min_y \left[\max(-(0.7125 - y_1 - 0.25x_{1,1} - 0.25x_{2,1}), 0)^6 \right. \\
& \quad + \max(-(2.5750 - y_2 - 0.25x_{1,2} - 0.25x_{2,2}), 0)^6 \\
& \quad + \max(-(y_3 + 0.25x_{1,3} + 0.25x_{2,3} - 0.9375), 0)^6 \\
& \quad \left. + \max(-(y_4 + 0.25x_{1,4} + 0.25x_{2,4} - 1.0250), 0)^6 \right].
\end{aligned}$$

6. kind = :nonlinear, fully expanded

Same structure, two substitutions:

- every squared term $(u - t)^2$ becomes $\cosh(u - t) - 1$;
- every constraint row h becomes $e^h - 1$.

The second substitution preserves the feasible set and the active set exactly, since $e^r - 1 = 0 \iff r = 0$ and $e^r - 1 > 0 \iff r > 0$. It does change the *geometry*: the rows are no longer affine, and $\cosh$ has nonvanishing derivatives of every order. Note $-(e^h - 1) = 1 - e^h$, which is how the penalty is written below.

Player 1

$$\begin{aligned}
& \min_{x_1 \in \mathbb{R}^5} \cosh(x_{1,5} - 0.80) - 1 \\
& \text{subject to } x_1 \in \mathcal{S}_1^{(2)}(x_2, x_3) \\
& \mathcal{S}_1^{(2)} = \arg \min_y [\cosh(y_3 - 0.15) - 1] + [\cosh(y_4 - 1.20) - 1] \quad \text{s.t. } y \in \mathcal{S}_1^{(3)}, \\
& \mathcal{S}_1^{(3)} = \arg \min_y [\cosh(y_1 - 1.00) - 1] + [\cosh(y_2 - 0.90) - 1] \quad \text{s.t. } y \in \mathcal{S}_1^{(4)}, \\
& \mathcal{S}_1^{(4)} = \arg \min_y \left[\max(1 - e^{0.6375 - y_1 - 0.25x_{2,1} - 0.25x_{3,1}}, 0)^6 \right. \\
& \quad + \max(1 - e^{2.4250 - y_2 - 0.25x_{2,2} - 0.25x_{3,2}}, 0)^6 \\
& \quad + \max(1 - e^{y_3 + 0.25x_{2,3} + 0.25x_{3,3} - 0.8625}, 0)^6 \\
& \quad \left. + \max(1 - e^{y_4 + 0.25x_{2,4} + 0.25x_{3,4} - 0.8750}, 0)^6 \right].
\end{aligned}$$

Player 2

$$\begin{aligned}
& \min_{x_2 \in \mathbb{R}^5} \cosh(x_{2,5} - 0.90) - 1 \\
& \text{subject to } x_2 \in \mathcal{S}_2^{(2)}(x_1, x_3)
\end{aligned}$$

$$\begin{aligned}
\mathcal{S}_2^{(2)} &= \arg \min_y [\cosh(y_3 - 0.20) - 1] + [\cosh(y_4 - 1.30) - 1] \quad \text{s.t.} \quad y \in \mathcal{S}_2^{(3)}, \\
\mathcal{S}_2^{(3)} &= \arg \min_y [\cosh(y_1 - 1.05) - 1] + [\cosh(y_2 - 1.00) - 1] \quad \text{s.t.} \quad y \in \mathcal{S}_2^{(4)}, \\
\mathcal{S}_2^{(4)} &= \arg \min_y \left[\max(1 - e^{0.6750 - y_1 - 0.25x_{1,1} - 0.25x_{3,1}}, 0)^6 \right. \\
&\quad + \max(1 - e^{2.5000 - y_2 - 0.25x_{1,2} - 0.25x_{3,2}}, 0)^6 \\
&\quad + \max(1 - e^{y_3 + 0.25x_{1,3} + 0.25x_{3,3} - 0.9000}, 0)^6 \\
&\quad \left. + \max(1 - e^{y_4 + 0.25x_{1,4} + 0.25x_{3,4} - 0.9500}, 0)^6 \right].
\end{aligned}$$

Player 3

$$\begin{aligned}
&\min_{x_3 \in \mathbb{R}^5} \quad \cosh(x_{3,5} - 1.00) - 1 \\
&\text{subject to} \quad x_3 \in \mathcal{S}_3^{(2)}(x_1, x_2) \\
\mathcal{S}_3^{(2)} &= \arg \min_y [\cosh(y_3 - 0.25) - 1] + [\cosh(y_4 - 1.40) - 1] \quad \text{s.t.} \quad y \in \mathcal{S}_3^{(3)}, \\
\mathcal{S}_3^{(3)} &= \arg \min_y [\cosh(y_1 - 1.10) - 1] + [\cosh(y_2 - 1.10) - 1] \quad \text{s.t.} \quad y \in \mathcal{S}_3^{(4)}, \\
\mathcal{S}_3^{(4)} &= \arg \min_y \left[\max(1 - e^{0.7125 - y_1 - 0.25x_{1,1} - 0.25x_{2,1}}, 0)^6 \right. \\
&\quad + \max(1 - e^{2.5750 - y_2 - 0.25x_{1,2} - 0.25x_{2,2}}, 0)^6 \\
&\quad + \max(1 - e^{y_3 + 0.25x_{1,3} + 0.25x_{2,3} - 0.9375}, 0)^6 \\
&\quad \left. + \max(1 - e^{y_4 + 0.25x_{1,4} + 0.25x_{2,4} - 1.0250}, 0)^6 \right].
\end{aligned}$$

7. Ground truth

The family is constructed so that the hard-constrained problem has the closed-form solution

$$x_1^* = \begin{pmatrix} 0.40 \\ 0.90 \\ 0.55 \\ 1.20 \\ 0.80 \end{pmatrix}, \quad x_2^* = \begin{pmatrix} 0.45 \\ 1.00 \\ 0.60 \\ 1.30 \\ 0.90 \end{pmatrix}, \quad x_3^* = \begin{pmatrix} 0.50 \\ 1.10 \\ 0.65 \\ 1.40 \\ 1.00 \end{pmatrix}.$$

`case.expected` in the test is this vector, flattened to length 15.

How it is constructed. t_p differs from x_p^* only in coordinates 1 and 3:

$$t_{p,1} = x_{p,1}^* + 0.6, \quad t_{p,3} = x_{p,3}^* - 0.4,$$

with $t_{p,2} = x_{p,2}^*$, $t_{p,4} = x_{p,4}^*$, $t_{p,5} = x_{p,5}^*$. Coordinates 2, 4 and 5 are therefore already at their unconstrained optimum and no constraint binds them. Coordinates 1 and 3 are pulled off target by exactly the two active rows: $C_{p,1}$ is *defined* as $c_{p,1}(x^*)$ and $F_{p,3}$ as $c_{p,3}(x^*)$, while the two inactive rows are offset by ± 1 .

Verification at x^* (identical for all three players):

row	$h_{p,m}(x^*),$:quadratic	$e^h - 1,$:nonlinear	status
1	0	0	active
2	1	$e - 1 \approx 1.71828$	inactive
3	0	0	active
4	1	$e - 1 \approx 1.71828$	inactive

Hence $P_p(x^*) = 0$ for every player: x^* attains the exact minimum of the highest-priority level. Since that level's argmin set equals the feasible set, the nested problem above has the *same* solution as the hard-constrained original — in exact arithmetic.

The 3×3 coupling matrix has 1 on the diagonal and 0.25 off it, so it is nonsingular and the two active rows pin each player's share individually, not just the aggregate.

8. Why solves do not land on x^*

x^* sits exactly on the boundary of the two active rows, where $h_{p,1} = h_{p,3} = 0$. The penalty $\max(-h, 0)^6$ has value *and first five derivatives all zero* there. The highest-priority level therefore exerts no gradient at the solution: nothing pulls an iterate back onto the active constraints, and the lower-priority objectives — which want coordinate 1 larger and coordinate 3 smaller — push it into mild infeasibility until the sixth power grows enough to resist.

Measured with the test's configuration (KLU, backtracking, $\text{tol} = 10^{-4}$, `max_outer_iters` = 1):

kind	z_0	$\ x - x^*\ _\infty$ reduced / quasi	max violation
:quadratic	expected	0.06068 / 0.06068	0.0910
:quadratic	zeros	0.72728 / 0.72728	0.0909
:quadratic	offset	0.06057 / 0.06057	0.0909
:nonlinear	expected	0.06257 / 0.06257	0.0896
:nonlinear	zeros	0.73006 / 0.73006	0.0907
:nonlinear	offset	0.06240 / 0.06240	0.0894

The residual is not what limits accuracy here — the degenerate penalty is. Tightening the tolerance to 10^{-6} makes every solve report `:failed` (the iteration stalls at $\|F\| \approx 1.5 \times 10^{-5} \dots 2.9 \times 10^{-5}$) yet moves the answer only from 0.061 to 0.046 away from x^* , with the violation shrinking from 0.091 to 0.069.

This gap is a property of the $(-h)^6$ encoding, not of the `:reduced` / `:quasi` choice: the two formulations agree with each other to between 3×10^{-15} and 4×10^{-6} , i.e. four to nine orders of magnitude closer to each other than either is to x^* .

It is the same encoding `robotic_arm_core.jl` uses for its collision and speed constraints, so the same slack should be expected there — a violation of 10^{-2} registers as a penalty of only 10^{-12} .